

Teaching Statement

Ridwan O. Olabiyi | Ph.D. Candidate, Industrial Engineering, Arizona State University | rolabiyi@asu.edu

From Data to Decisions in Engineered Systems

My teaching philosophy is organized around a simple objective: students should leave an engineering course able to use a model, explain why it is appropriate, test its assumptions, and translate its output into a defensible decision. This philosophy grew from teaching mathematics and physics in Nigeria and developed further through instructional responsibilities in Industrial Engineering at Arizona State University, including Probability and Statistics for Engineering Problem Solving and Advanced Quality Control. Across these settings, I have learned that technical confidence grows when abstract ideas are made observable, complex reasoning is broken into deliberate steps, and assessment rewards the quality of the problem-solving process rather than only the final numeric answer. I therefore structure teaching around three practices: make models tangible and purposeful, scaffold iterative reasoning, and use authentic engineering problems to develop judgment.

1. Make abstract models tangible and purposeful

Probability, statistics, optimization, and machine learning can quickly become collections of formulas unless students can see the physical or operational meaning behind them. I begin with the system and the decision before introducing the formal method: What is being observed? What is uncertain? What decision must be made? What assumptions make the model reasonable? In probability and statistics, this shifts attention from selecting a formula to deciding what a probability model represents and how variability affects an engineering conclusion. In quality engineering, it turns a control-chart statistic into a question about whether a process has meaningfully changed and what action should follow.

My experience with Advanced Quality Control reinforced the value of this approach. Student teams investigated modern statistical monitoring methods and were expected to explain the method, reproduce its logic, and connect it to an engineering context. One project examined Gaussian-mixture-model change-point detection through a semiconductor process-monitoring example in which wafer thickness transitions between operating states. The exercise required students to move among statistical assumptions, algorithmic steps, implementation, and the practical consequence of declaring a process shift. Projects of this kind make advanced methods less opaque because students can trace each mathematical object to a system behavior and a decision.

2. Scaffold reasoning, then transfer ownership

Students develop independence when expert reasoning is made visible and support is removed deliberately. I organize difficult problems into a repeatable sequence: formulate the engineering question; identify knowns, unknowns, and sources of uncertainty; select a model and state its assumptions; compute or simulate; validate the result; and interpret the answer in the original context. Early assignments provide prompts for these steps. Later assignments provide only the problem statement, requiring students to decide what tools and checks are necessary. This structure is especially useful in computational courses because it prevents Python or software output from becoming a substitute for engineering reasoning.

Assessment follows the same principle. I value a correct result, but I also evaluate model choice, assumptions, units, diagnostic checks, interpretation, and communication of uncertainty. Low-stakes checkpoints reveal misconceptions before high-stakes assessments. For larger projects, I use staged deliverables such as problem formulation, preliminary analysis, code or model review, a final technical report, and presentation. In advanced courses, reproducible notebooks, version control, and simple automated tests can make computational reasoning auditable while exposing students to professional engineering practice.

3. Use authentic, open-ended engineering problems

Open-ended projects provide a bridge between classroom knowledge and the ambiguity students will face as engineers. I want students to encounter incomplete data, competing objectives, model mismatch, and the need to justify a recommendation. In a planned junior-level Manufacturing Data Analytics course, teams would collect streaming vibration or temperature measurements from a benchtop process and build a statistical monitoring or prediction pipeline in Python. Instead of ending with a single exam, teams would produce a short conference-style paper and presentation explaining the engineering problem, model, uncertainty, limitations, and recommended action. The format develops statistical thinking, scientific writing, teamwork, and the habit of treating data analysis as an engineering argument rather than a software exercise.

At the graduate level, I plan a course in Physics-Informed Machine Learning and Uncertainty-Aware Digital Twins. Students would construct a reduced-order, neural-operator, or sparse-identification model for a physical or cyber-physical system; calibrate it using data; quantify predictive uncertainty; and decide when the model should be updated or rejected. A final project would require each team to demonstrate an end-to-end loop involving a physical or simulated system, sensing, model calibration, prediction, validation, and a decision or control action. This course would connect statistical inference and machine learning to the physical reasoning expected of engineers.

Welcoming and inclusive learning through transparent structure

A welcoming classroom is one in which expectations are explicit and students have multiple legitimate ways to demonstrate competence. I use transparent rubrics, worked examples that emphasize reasoning rather than hidden tricks, and early diagnostic opportunities so students who enter with different preparation levels can identify what they need to strengthen. Office hours and project check-ins are normal components of the learning process rather than remediation reserved for students who are struggling. In team projects, rotating technical and communication roles distribute responsibility and reduce the likelihood that one student becomes the permanent programmer, analyst, or presenter. Written, verbal, visual, and computational modes of explanation give students several ways to engage while preserving common technical standards.

My mentoring follows the same logic. I have mentored undergraduate researchers through ASU summer programs on machine learning for CAD model generation and active learning. My future laboratory will use a tiered structure in which experienced graduate researchers mentor junior students with clear milestones, reproducible workflows, and visible contribution records. I also intend to create remote research opportunities for students who have limited access to research-intensive environments. The purpose is to make the pathway into research visible: begin with a tractable problem, receive regular feedback, learn reproducible tools, present results early, and gradually assume intellectual ownership.

Teaching contributions and continuous improvement

I am prepared to teach undergraduate and graduate courses in probability and statistics, quality and reliability engineering, data analytics, engineering simulation, machine learning, manufacturing systems, uncertainty quantification, and systems modeling. My research also creates natural opportunities to develop specialized instruction in scientific machine learning, digital twins, physics discovery, and data-driven engineering. Regardless of course level, I aim to connect mathematical rigor to observable systems and to require students to explain not only how an answer was obtained, but why it should be trusted and what decision it supports.

I treat teaching as an iterative engineering process. Course effectiveness should be evaluated with evidence such as concept-level diagnostics, performance on common rubric elements, quality of technical explanations, project completion, and structured student feedback. I use these signals to identify where students are memorizing rather than reasoning and to revise examples, scaffolds, and assessments accordingly. My goal is a classroom in which students become increasingly independent at formulating problems, selecting models, testing assumptions, communicating uncertainty, and making defensible engineering decisions. That habit of disciplined reasoning is the central teaching contribution I hope to bring to my future faculty role.